॥ श्री गणेशाय नमः ॥

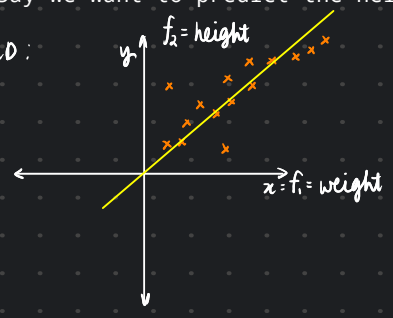
Linear Regression

regression technique

$D = \langle x_i, y_i \rangle_{i=1}^{n}$ $\quad x_i \in \mathbb{R}^n,\ y \in \mathbb{R}$ (ordinary least squares / linear least squares)

Say we want to predict the height of a person, given their weight, gender, ethnicity, and hair color.

2D:



The goal of linear regression is to find a line that fits the given data as well as possible.

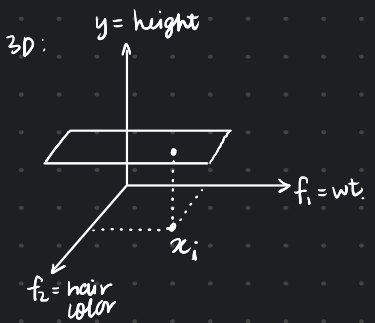
height = w_1 * weight + w_0

$y = mx + c$

wt. hairscolor

$x_i = \langle x_{i1}, x_{i2} \rangle$

3D:



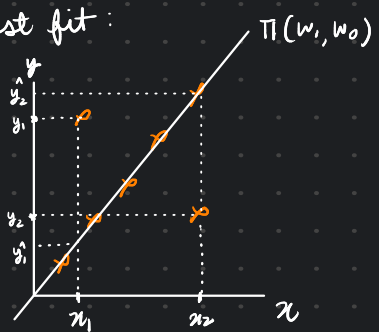
height = $w_1 f_1 + w_2 f_2 + w_0$

$y_i = w_1 x_{i1} + w_2 x_{i2} + w_0$

$\downarrow$

$y_i = w^T x_i + w_0$ ⟵ hyperplane.

Best fit:



$f(x) = w_1 x + w_0$

let $f(x_1) = \hat{y}_1 \neq y_1$

$f(x_2) = \hat{y}_2 \neq y_2$

error for x_1 $= y_1 - \hat{y}_1$

error for x_2 $= y_2 - \hat{y}_2$

error for a point on line $= y_i - \hat{y}_i = 0$

We want to get the min. sum of errors.

$error_i = y_i - \hat{y}_i$ (could be +ve/-ve based on position w.r.t. line)

So we consider squared error.

$$(w^*, w_0^*) = \underset{w, w_0}{\operatorname{argmin}} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

(w: vector, w_0: scalar)

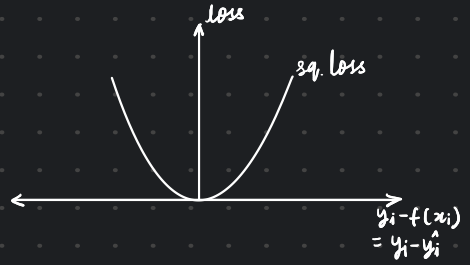
$$= \underset{w, w_0}{\operatorname{argmin}} \sum_{i=1}^{n} \underbrace{\left[y_i - (w^T x_i + x_0) \right]^2}_{\text{sq. loss}}$$

$\hat{y}_i = f(x_i) = w^T x_i + x_0$

Regularization

$$(w^*, w_0^*) = \underset{w, w_0}{\operatorname{argmin}} \sum_{i=1}^{n} \left[y_i - (w^T x_i + w_0) \right]^2 + \underbrace{\lambda \|w\|_2^2}_{L_2 \text{ regularizn}}$$

As loss minimization:



Cases

1. Imbalanced data : upsampling/downsampling
2. Feature importance and interpretability :
   If features are not multi-collinear → use $|w_j|$ (same as logistic regression)
3. Feature Engineering / Transforms : same
4. Regularization : same

   L_1 : $\lambda \uparrow$ → sparsity ↑
   feature imp ↑
5. outliers : not the same as logistic regression.
   ↳ there $\sigma(x)$ limited the effect of outliers.

   Here, say a point has high $|y_i - \hat{y}_i|$

   $(y_i - \hat{y}_i) \uparrow$ then $(y_i - \hat{y}_i)^2 \uparrow\uparrow$

   Sq. error will be hugely impacted.

   soln : We use D_{train} to find (w^*, w_0^*).
   We then find points really far from the hyperplane. ($y_i - \hat{y}_i$ is high)
   Remove those points.

   $D'_{train} = D_{train} - \{outliers\}$

   Now find a hyperplane on D'_{train}.

   You can do this iteratively. This is called RANSAC.
   Random Sampling Consensus.

```
from sklearn.datasets import load_boston
boston = load_boston()
```

```
print(boston.data.shape)
```

```
(506, 13)
```

$y_i \rightarrow$ house prices

```
print(boston.feature_names)
```

```
['CRIM' 'ZN' 'INDUS' 'CHAS' 'NOX' 'RM' 'AGE' 'DIS' 'RAD' 'TAX' 'PTRATIO'
 'B' 'LSTAT']
```

```
print(boston.target)
```

```
[ 24.   21.6  34.7  33.4  36.2  28.7  22.9  27.1  16.5  18.9  15.   18.9
  21.7  20.4  18.2  19.9  23.1  17.5  20.2  18.2  13.6  19.6  15.2  14.5
  15.6  13.9  16.6  14.8  18.4  21.   12.7  14.5  13.2  13.1  13.5  18.9
  ...
```

```
print(boston.DESCR)
```

```
Boston House Prices dataset
===========================

Notes
------
Data Set Characteristics:

    :Number of Instances: 506

    :Number of Attributes: 13 numeric/categorical predictive

    :Median Value (attribute 14) is usually the target

    :Attribute Information (in order):
        - CRIM     per capita crime rate by town
        - ZN       proportion of residential land zoned for lots over 25,
000 sq.ft.
        - INDUS    proportion of non-retail business acres per town
        - CHAS     Charles River dummy variable (= 1 if tract bounds rive
r; 0 otherwise)
        - NOX      nitric oxides concentration (parts per 10 million)
        - RM       average number of rooms per dwelling
        - AGE      proportion of owner-occupied units built prior to 1940
        - DIS      weighted distances to five Boston employment centres
        - RAD      index of accessibility to radial highways
        - TAX      full-value property-tax rate per $10,000
        - PTRATIO  pupil-teacher ratio by town
        - B        1000(Bk - 0.63)^2 where Bk is the proportion of blacks
by town
        - LSTAT    % lower status of the population
        - MEDV     Median value of owner-occupied homes in $1000's

    :Missing Attribute Values: None

    :Creator: Harrison, D. and Rubinfeld, D.L.

This is a copy of UCI ML housing dataset.
http://archive.ics.uci.edu/ml/datasets/Housing


This dataset was taken from the StatLib library which is maintained at Ca
rnegie Mellon University.

The Boston house-price data of Harrison, D. and Rubinfeld, D.L. 'Hedonic
prices and the demand for clean air', J. Environ. Economics & Management,
vol.5, 81-102, 1978.   Used in Belsley, Kuh & Welsch, 'Regression diagnos
tics
...', Wiley, 1980.   N.B. Various transformations are used in the table o
n
pages 244-261 of the latter.

The Boston house-price data has been used in many machine learning papers
 that address regression
problems.

**References**

   - Belsley, Kuh & Welsch, 'Regression diagnostics: Identifying Influent
ial Data and Sources of Collinearity', Wiley, 1980. 244-261.
   - Quinlan,R. (1993). Combining Instance-Based and Model-Based Learnin
g. In Proceedings on the Tenth International Conference of Machine Learni
ng, 236-243, University of Massachusetts, Amherst. Morgan Kaufmann.
   - many more! (see http://archive.ics.uci.edu/ml/datasets/Housing)
```

```
import pandas as pd
bos = pd.DataFrame(boston.data)
print(bos.head())
```

```
        0     1     2    3      4      5     6       7    8      9     10
  \
0  0.00632  18.0  2.31  0.0  0.538  6.575  65.2  4.0900  1.0  296.0  15.3

1  0.02731   0.0  7.07  0.0  0.469  6.421  78.9  4.9671  2.0  242.0  17.8

2  0.02729   0.0  7.07  0.0  0.469  7.185  61.1  4.9671  2.0  242.0  17.8

3  0.03237   0.0  2.18  0.0  0.458  6.998  45.8  6.0622  3.0  222.0  18.7

4  0.06905   0.0  2.18  0.0  0.458  7.147  54.2  6.0622  3.0  222.0  18.7
```

```
        11    12
0   396.90  4.98
1   396.90  9.14
2   392.83  4.03
3   394.63  2.94
4   396.90  5.33
```

```
bos['PRICE'] = boston.target

X = bos.drop('PRICE', axis = 1)
Y = bos['PRICE']
```

```
import sklearn
X_train, X_test, Y_train, Y_test = sklearn.cross_validation.train_test_spli
print(X_train.shape)
print(X_test.shape)
print(Y_train.shape)
print(Y_test.shape)
```

```
(339, 13)
(167, 13)
(339,)
(167,)
```

```
# code source:https://medium.com/@haydar_ai/learning-data-science-day-9-lin
from sklearn.linear_model import LinearRegression

lm = LinearRegression()
lm.fit(X_train, Y_train)

Y_pred = lm.predict(X_test)

plt.scatter(Y_test, Y_pred)
plt.xlabel("Prices: $Y_i$")
plt.ylabel("Predicted prices: $\hat{Y}_i$")
plt.title("Prices vs Predicted prices: $Y_i$ vs $\hat{Y}_i$")
plt.show()
```